

Módulo 08 - Painel Administrativo e Agenda Operacional

Objetivo

Estudar a evolução do painel administrativo: Django Admin, telas Next.js, proteção staff, listagem e detalhe de pedidos, ações operacionais e agenda semanal.

Commits Estudados

- `6aa031a` - telas iniciais do painel admin ainda sem API.
- `7140aaa` - commit repetido sem diff material.
- `0083e30` - melhora usabilidade inicial do Django Admin.
- `52b1c7a` - actions seguras no Django Admin.
- `d262675` - adiciona listagem e detalhe admin de pedidos no backend.
- `1b5a814` - conecta listagem admin de pedidos.
- `b4ba3c7` - protege painel admin por sessão staff.
- `059c913` - permite login staff sem cliente.
- `8f0b85a` - conecta detalhe admin de pedido real.
- `b60d932` - conecta ações admin de pedido.
- `b1f3b0a` - corrige login frontend para staff sem cliente.
- `bf44c80` - corrige redirecionamento pós-login admin.
- `65528b4` - corrige guard do painel admin.
- `a9be2e2` - corrige logout de sessões Django.
- `33f8c10` - corrige ações disponíveis de pedido.
- `74b69c7` - cria tela semanal de agenda operacional.
- `dba6094` - adiciona endpoint backend da agenda.
- `02a9e10` - melhora visual da agenda operacional.

Aula 1 - Telas Admin Antes da API

O commit `6aa031a` adiciona telas do painel admin ainda sem conexão com API.

Código para ler:

- `frontend/src/app/admin/page.tsx`
- `frontend/src/app/admin/categorias/page.tsx`
- `frontend/src/app/admin/entregas/page.tsx`
- `frontend/src/app/admin/kits/page.tsx`
- `frontend/src/app/login/page.tsx`

Ponto real:

- As telas existem como superfície de produto.
- O commit deixa claro no título que ainda falta conexão e revisão.
- Isso ajuda a separar prototipação visual de contrato backend.

O commit `7140aaa` aparece na história com o mesmo título e sem diff material. Para estudo técnico, ele serve apenas como registro histórico, não como aula de código.

Aula 2 - Django Admin Como Ferramenta Operacional

O commit `0083e30` melhora usabilidade inicial do Django Admin.

Código para ler:

- `backend/catalogo/admin.py`
- `backend/clientes/admin.py`
- `backend/entregas/admin.py`
- `backend/pedidos/admin.py`

Melhorias reais:

- `list_display`;
- filtros;
- busca;
- organização de campos;
- inlines para relações importantes.

Ponto de engenharia:

- Antes de ter painel customizado completo, Django Admin entrega operação real para dados internos.
- Admin não substitui regra de negócio no service.

Aula 3 - Actions Seguras no Django Admin

O commit `52b1c7a` adiciona actions seguras.

Código para ler:

- `backend/catalogo/admin.py`
- `backend/pedidos/admin.py`
- `backend/catalogo/tests.py`
- `backend/pedidos/tests.py`

Ponto real:

- Actions chamam services em vez de reimplementar regra dentro do admin.
- Erros são reportados sem interromper todo o lote.
- Campos críticos ficam readonly.

Testes para estudar:

- `test_admin_action_libera_unidades_usando_service_e_reporta_falhas`
- `test_action_reservar_unidades_chama_service_e_reporta_falha_por_status`
- `test_action_confirmar_pedidos_continua_apos_erro_do_service`

Lição:

- Django Admin também precisa de testes quando dispara regra crítica.

Aula 4 - API Admin de Pedidos

O commit `d262675` cria listagem e detalhe admin no backend.

O commit `1b5a814` conecta a listagem no frontend.

Código backend:

- `backend/pedidos/views.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/tests.py`

- `backend/pedidos/urls.py`

Código frontend:

- `frontend/src/app/admin/pedidos/page.tsx`
- `frontend/src/services/adminPedidos.ts`
- `frontend/src/types/adminPedidos.ts`

Endpoint:

- `GET /api/admin/pedidos/`

Recursos reais:

- paginação;
- busca por nome, e-mail, telefone e id;
- filtro por status;
- ordenação permitida;
- serializer resumido para listagem;
- `IsAdminUser` .

Testes para estudar:

- `test_anonimo_nao_acessa_lista_admin`
- `test_usuario_comum_nao_acessa_lista_admin`
- `test_admin_lista_pedidos_com_campos_resumidos`
- `test_filtro_por_status_funciona`
- `test_busca_por_nome_email_e_telefone_snapshot_funciona`

Aula 5 - Detalhe Admin Real

O commit `8f0b85a` conecta o detalhe admin de pedido.

Código para ler:

- `frontend/src/app/admin/pedidos/[id]/page.tsx`
- `frontend/src/services/adminPedidos.ts`
- `frontend/src/types/adminPedidos.ts`
- `backend/pedidos/serializers.py`

Endpoint:

- `GET /api/admin/pedidos/<id>/`

Dados reais exibidos:

- cliente;
- snapshot do cliente;
- endereço;
- valores;
- itens;
- aceite de contrato;
- reservas;
- unidades reservadas;
- ações disponíveis.

Ponto de TypeScript:

- `AdminPedidoDetail` precisa representar um JSON complexo vindo do serializer admin.
- Isso transforma o serializer backend em contrato explícito para a tela.

Aula 6 - Ações Admin Conectadas

O commit `b60d932` conecta ações admin de pedido no frontend.

Código para ler:

- `frontend/src/app/admin/pedidos/[id]/page.tsx`
- `frontend/src/services/adminPedidos.ts`
- `frontend/src/types/adminPedidos.ts`
- `backend/pedidos/views.py`
- `backend/pedidos/services.py`

Ações conectadas:

- reservar unidades;
- confirmar pedido;
- iniciar locação;
- registrar retirada.

Endpoints:

- `POST /api/admin/pedidos/<id>/reservar-unidades/`
- `POST /api/admin/pedidos/<id>/confirmar/`
- `POST /api/admin/pedidos/<id>/iniciar-locacao/`
- `POST /api/admin/pedidos/<id>/registrar-retirada/`

Ponto crítico:

- Frontend só dispara a ação.
- Backend valida status, contrato, reserva, unidade e transação.

Aula 7 - Guard do Painel Admin

Os commits `b4ba3c7`, `059c913`, `b1f3b0a`, `bf44c80`, `65528b4` e `a9be2e2` endurecem o fluxo de acesso admin.

Código para ler:

- `backend/clientes/views.py`
- `backend/clientes/serializers.py`
- `frontend/src/components/admin/AdminLayout/index.tsx`
- `frontend/src/contexts/AuthContext.tsx`
- `frontend/src/app/login/page.tsx`
- `frontend/src/services/auth.ts`

Problemas reais corrigidos:

- staff sem cliente precisa conseguir logar;
- cliente comum não deve acessar painel;
- redirecionamento pós-login admin precisa respeitar destino;
- guard não pode liberar tela antes de confirmar staff;
- logout precisa limpar acesso admin.

Testes backend para estudar:

- `test_admin_me_anonimo_negado_com_401`
- `test_admin_me_cliente_comum_negado_com_403`
- `test_admin_me_staff_permitido`
- `test_logout_staff_sem_cliente_encerra_auth_e_acesso_admin`

Aula 8 - Ações Disponíveis São Regra de Produto

O commit `33f8c10` corrige ações disponíveis.

Código para ler:

- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/tests.py`

Service atual:

```
class AdminPedidoAcoesService:
    @staticmethod
    def acoes_disponiveis(pedido):
        if pedido.status == Pedido.Status.AGUARDANDO_ANALISE:
            ...
        if pedido.status == Pedido.Status.RESERVADO:
            ...
```

Lição:

- UI não deve adivinhar transição de estado.
- Backend responde quais ações são válidas naquele momento.
- Isso reduz erro operacional.

Aula 9 - Agenda Operacional

O commit `74b69c7` cria a tela semanal da agenda no frontend.

O commit `dba6094` adiciona o endpoint backend.

O commit `02a9e10` melhora o visual da agenda.

Código backend:

- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/views.py`
- `backend/pedidos/urls.py`
- `backend/pedidos/tests.py`

Código frontend:

- `frontend/src/app/admin/agenda/page.tsx`
- `frontend/src/services/adminAgenda.ts`
- `frontend/src/types/adminAgenda.ts`
- `frontend/src/components/admin/AdminSideBar/index.tsx`

Endpoint:

- `GET /api/admin/agenda/?inicio=YYYY-MM-DD&fim=YYYY-MM-DD`

Eventos reais:

- entrega;
- retirada;
- contrato pendente;
- locação em andamento.

Regras reais:

- agenda exige staff;
- intervalo máximo é 31 dias;
- tipo inválido retorna erro;
- dados sensíveis do aceite não são expostos;
- eventos são ordenados por data, tipo e pedido.

Testes para estudar:

- `test_admin_lista_eventos_operacionais_do_periodo`
- `test_agenda_filtrar_por_tipo`
- `test_agenda_filtrar_por_status_do_pedido`
- `test_locacao_em_andamento_ancora_no_inicio_do_periodo_consultado`
- `test_agenda_nao_expoe_dados_sensíveis_do_aceite`

Revisão do Módulo

Você deve conseguir responder:

- Por que Django Admin ainda é relevante mesmo com painel Next.js?
- Onde o backend protege rotas admin?

- Por que actions admin chamam services?
- Como a tela de detalhe sabe quais ações exibir?
- Que dados a agenda operacional agrega e quais dados ela não deve expor?

Exercício Prático

Abra `git show dba6094` e siga:

1. query serializer valida `inicio`, `fim`, `tipo` e `status`;
2. service gera eventos;
3. view exige `IsAdminUser`;
4. tests garantem segurança e formato.

Depois compare com `git show 74b69c7` e veja como o frontend esperava consumir a agenda.